

Operating Systems

CS-2018

Semester Project

Spring 2026



Real-Time Scheduling Simulator: Industry Applications Electric Vehicle Battery Management System (BMS)

Submitted by:

Muhammad Ibrahim (23i – 6090)

Mohammad Ahtesam (23i – 6060)

Section:

A

Date of Submission:

10/06/2026

Course Instructor:

Engr. Shehzad Ahmed

Lab Instructor:

Engr. Ammar Ali

National University of Computer and Emerging Sciences, Islamabad

Abstract

This project is a simulation of a real-time Battery Management System (BMS). A BMS controls the high-voltage battery pack in an electric vehicle. It needs to run several tasks at the same time, such as checking cell voltage, measuring temperature, calculating battery percentage, balancing cells, and logging data. To make sure the battery stays safe, we used an algorithm called Rate Monotonic Scheduling (RMS). This algorithm gives the highest priority to the tasks that need to run most frequently. The project is written in C++ using POSIX threads (pthreads) to run the tasks and mutex locks to keep the timing accurate. The simulation runs for 2000ms. The results show that all high-priority tasks run exactly when they are supposed to, no deadlines are missed, and the total CPU usage stays at 55%.

Industry Scenario Description

In the electric vehicle industry, the Battery Management System (BMS) is the most important electronics component for keeping the battery pack safe. A lithium-ion battery pack is made of hundreds of small battery cells connected together. If these cells get too hot or have too much voltage, they can permanently break or even catch on fire.

To keep the vehicle safe, the BMS has to do many jobs at different speeds. For example, it must check cell voltage very fast so it can stop the system if there is a sudden short circuit. It also needs to check the temperature frequently to turn on the cooling fans. Other tasks, like calculating the remaining driving range or saving history logs to memory, can run much slower.

In a real vehicle, we cannot use a simple scheduling system where tasks just take turns. If the computer is busy saving a large history log and delays checking the temperature, the battery could overheat. This is why a real-time operating system (RTOS) is required. A real-time scheduler guarantees that high-priority safety tasks will immediately interrupt slower, less important tasks. This makes sure that critical safety data is always processed on time.

Chosen Scheduling Algorithm

Rate Monotonic Scheduling (RMS) is a real-time scheduling algorithm where priorities are assigned based on time periods. The rule is very simple: tasks with shorter periods get higher priority, and tasks with longer periods get lower priority. It is also a pre-emptive algorithm, which means if a high priority task suddenly needs to run, it will immediately stop any lower priority task that is currently using the CPU.

RMS is perfect for a Battery Management System because sensor data comes in at fixed, regular times. By using RMS, we make sure that fast safety checks like checking voltage every 100ms always run before slow background jobs, like saving logs every 2000ms.

To prove that our 5 tasks can run safely without missing their deadlines, we use a standard mathematical formula called the Liu and Layland bound:

$$U = n \left(2^{\frac{1}{n}} - 1 \right)$$

In this formula, n stands for the number of tasks. Since we have 5 tasks, the math looks like this:

$$U = 5 \left(2^{\frac{1}{5}} - 1 \right) \approx 0.693$$

This answer means that if our total CPU usage is less than 69.3%, the scheduler is mathematically guaranteed to never miss a deadline.

When we ran our simulation, our actual CPU utilization was 55%. Since 55% is well below the 69.3% limit, our project proves mathematically that the battery tasks will always finish on time.

System Design

Our system is designed using a clear, object-oriented approach in C++. The design is split into two main parts: the Task structure and the Scheduler logic.

Task Structure

Each individual job in the Battery Management System is represented by a Task object. The class stores all the data needed to track and schedule the job. The variables inside each task are:

- **Name:** A text string to identify the task (like "Voltage Monitor").
- **Period:** How often the task needs to run (in milliseconds).
- **Execution Time:** How much CPU time the task needs to finish its job during each period.
- **Priority:** A number representing the task priority under RMS rules (smaller numbers mean higher priority).
- **Remaining Time:** Tracks how many milliseconds of execution time are left for the task in the current period.
- **Next Deadline:** The exact global time when the current task must be finished.

The class also uses simple getter and setter functions so the scheduler can safely check and update these variables during the simulation.

Scheduler Logic

The core scheduler works as a digital clock simulation that advances by a fixed step size called a "tick" (set to 10ms). The entire scheduling logic follows these steps inside a loop:

1. **Check for New Periods:** At every tick, the scheduler checks if the current global time matches the period of any task. If a period arrives, the task resets its remaining execution time back to its full execution time, and its next deadline is extended.
2. **Deadline Miss Check:** Before resetting the task, if the task still has remaining execution time from its previous period, it means it missed its deadline, and a warning is logged.
3. **Find the Highest Priority Task:** The scheduler loops through all 5 tasks to find which ones have remaining execution time left. Out of those ready tasks, it selects the one with the highest priority (the lowest priority value).

4. **Execute the Task:** The chosen task runs for 10ms (one tick), and its remaining execution time is reduced by 10ms.
5. **CPU Idle Check:** If no tasks have any remaining execution time left, the CPU stays idle for that tick, and an idle counter is increased.
6. **Advance Time:** The global time increases by 10ms, and the loop repeats until the total simulation duration (2000ms) is reached.

Implementation Details

The simulator is implemented using standard Linux programming techniques in C++. It relies on three main concepts: thread creation, synchronization, and preemption logic.

Thread Creation

When the program starts, it initializes 5 separate threads using the POSIX threads library (pthread.h). Each thread represents one of our Battery Management System tasks. We use a loop in the main() function and call pthread_create() to launch all 5 threads simultaneously:

```
for(int i = 0; i < 5; i++){  
    pthread_create(&threads[i], NULL, simulateTask, (void*)&bmsTasks[i]);  
}
```

Each thread runs the simulateTask function, which prints an initialization message and then safely exits. In a real operating system, these threads would run continuously, but here they represent the active hardware modules of our system. At the end of main(), we use pthread_join() to safely clean up and close all 5 threads to prevent memory leaks.

```
for(int i = 0; i < 5; i++){  
    pthread_join(threads[i], NULL);  
}
```

Synchronization

Because multiple threads and the main scheduler loop run at the same time, they share access to the task data. To prevent data corruption, we use a mutex lock (pthread_mutex_t).

Before any thread prints to the screen or before the scheduler checks task variables, it must lock the mutex using pthread_mutex_lock(&taskMutex). Once the work is done, it releases the lock using pthread_mutex_unlock(&taskMutex). This ensures that only one part of the program can modify the system state at any exact moment.

Preemption Logic

Preemption means a more important task can interrupt a less important task. In our implementation, this is handled mathematically inside the main simulation loop.

Every 10ms (one simulation tick), the scheduler checks all tasks. Even if a low-priority task (like the Logger) is currently running and has remaining time left, the scheduler will instantly switch the CPU to a high-priority task (like the Voltage Monitor) if its new period arrives. The

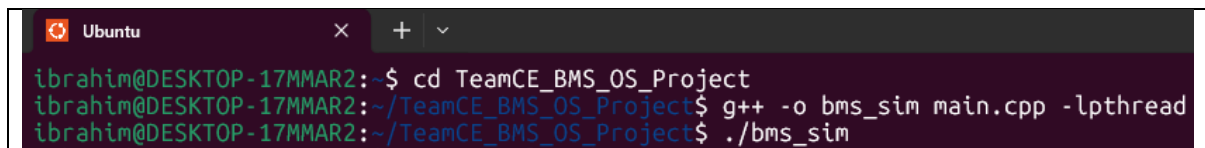
scheduler achieves this by selecting the task with the lowest priority number value at every single tick, creating a perfect simulation of real-time preemption.

Execution Output

This section presents the actual terminal output from our simulation. The logs demonstrate the exact order of task execution, how preemption works in real-time, and how the performance metrics are tracked.

Terminal Compilation and Run Commands

We navigated to our project directory, compiled the code using the POSIX threads flag, and executed. The exact terminal commands used are:

A terminal window with a dark background and light text. The prompt is 'ibrahim@DESKTOP-17MMAR2:~\$'. The first command is 'cd TeamCE_BMS_OS_Project'. The second command is 'g++ -o bms_sim main.cpp -lpthread'. The third command is './bms_sim'.

```
ibrahim@DESKTOP-17MMAR2:~$ cd TeamCE_BMS_OS_Project
ibrahim@DESKTOP-17MMAR2:~/TeamCE_BMS_OS_Project$ g++ -o bms_sim main.cpp -lpthread
ibrahim@DESKTOP-17MMAR2:~/TeamCE_BMS_OS_Project$ ./bms_sim
```

Simulation Startup and Initialization Log

When the program launches, all 5 threads are created. Each thread locks the mutex, prints its initialization message, and releases the mutex.

(Code Pasted from Output File)

```
Starting RMS Scheduler Simulation...
Thread 'Voltage Monitor' is initialized!
Thread 'Temperature Sensor' is initialized!
Thread 'SoC Calculator' is initialized!
Thread 'Cell Balancer' is initialized!
Thread 'Logger' is initialized!
```

Step-by-Step Execution Trace

Here is the timeline log from 0ms to 1990ms. It clearly shows our Rate Monotonic priority logic in action:

(Code Pasted from Output File)

```
[Time: 0ms] Running: 'Voltage Monitor' (Remaining: 20ms)
[Time: 10ms] Running: 'Voltage Monitor' (Remaining: 10ms)
[Time: 20ms] Running: 'Temperature Sensor' (Remaining: 20ms)
[Time: 30ms] Running: 'Temperature Sensor' (Remaining: 10ms)
[Time: 40ms] Running: 'SoC Calculator' (Remaining: 50ms)
[Time: 50ms] Running: 'SoC Calculator' (Remaining: 40ms)
[Time: 60ms] Running: 'SoC Calculator' (Remaining: 30ms)
[Time: 70ms] Running: 'SoC Calculator' (Remaining: 20ms)
[Time: 80ms] Running: 'SoC Calculator' (Remaining: 10ms)
[Time: 90ms] Running: 'Cell Balancer' (Remaining: 100ms)
[Time: 100ms] Running: 'Voltage Monitor' (Remaining: 20ms)
[Time: 110ms] Running: 'Voltage Monitor' (Remaining: 10ms)
[Time: 120ms] Running: 'Cell Balancer' (Remaining: 90ms)
```

[Time: 130ms] Running: 'Cell Balancer' (Remaining: 80ms)
 [Time: 140ms] Running: 'Cell Balancer' (Remaining: 70ms)
 [Time: 150ms] Running: 'Cell Balancer' (Remaining: 60ms)
 [Time: 160ms] Running: 'Cell Balancer' (Remaining: 50ms)
 [Time: 170ms] Running: 'Cell Balancer' (Remaining: 40ms)
 [Time: 180ms] Running: 'Cell Balancer' (Remaining: 30ms)
 [Time: 190ms] Running: 'Cell Balancer' (Remaining: 20ms)
 [Time: 200ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 210ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 220ms] Running: 'Temperature Sensor' (Remaining: 20ms)
 [Time: 230ms] Running: 'Temperature Sensor' (Remaining: 10ms)
 [Time: 240ms] Running: 'Cell Balancer' (Remaining: 10ms)
 [Time: 250ms] Running: 'Logger' (Remaining: 100ms)
 [Time: 260ms] Running: 'Logger' (Remaining: 90ms)
 [Time: 270ms] Running: 'Logger' (Remaining: 80ms)
 [Time: 280ms] Running: 'Logger' (Remaining: 70ms)
 [Time: 290ms] Running: 'Logger' (Remaining: 60ms)
 [Time: 300ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 310ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 320ms] Running: 'Logger' (Remaining: 50ms)
 [Time: 330ms] Running: 'Logger' (Remaining: 40ms)
 [Time: 340ms] Running: 'Logger' (Remaining: 30ms)
 [Time: 350ms] Running: 'Logger' (Remaining: 20ms)
 [Time: 360ms] Running: 'Logger' (Remaining: 10ms)
 [Time: 370ms] CPU Idle
 [Time: 380ms] CPU Idle
 [Time: 390ms] CPU Idle
 [Time: 400ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 410ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 420ms] Running: 'Temperature Sensor' (Remaining: 20ms)
 [Time: 430ms] Running: 'Temperature Sensor' (Remaining: 10ms)
 [Time: 440ms] CPU Idle
 [Time: 450ms] CPU Idle
 [Time: 460ms] CPU Idle
 [Time: 470ms] CPU Idle
 [Time: 480ms] CPU Idle
 [Time: 490ms] CPU Idle
 [Time: 500ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 510ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 520ms] Running: 'SoC Calculator' (Remaining: 50ms)
 [Time: 530ms] Running: 'SoC Calculator' (Remaining: 40ms)
 [Time: 540ms] Running: 'SoC Calculator' (Remaining: 30ms)
 [Time: 550ms] Running: 'SoC Calculator' (Remaining: 20ms)
 [Time: 560ms] Running: 'SoC Calculator' (Remaining: 10ms)
 [Time: 570ms] CPU Idle
 [Time: 580ms] CPU Idle
 [Time: 590ms] CPU Idle
 [Time: 600ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 610ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 620ms] Running: 'Temperature Sensor' (Remaining: 20ms)

[Time: 630ms] Running: 'Temperature Sensor' (Remaining: 10ms)
 [Time: 640ms] CPU Idle
 [Time: 650ms] CPU Idle
 [Time: 660ms] CPU Idle
 [Time: 670ms] CPU Idle
 [Time: 680ms] CPU Idle
 [Time: 690ms] CPU Idle
 [Time: 700ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 710ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 720ms] CPU Idle
 [Time: 730ms] CPU Idle
 [Time: 740ms] CPU Idle
 [Time: 750ms] CPU Idle
 [Time: 760ms] CPU Idle
 [Time: 770ms] CPU Idle
 [Time: 780ms] CPU Idle
 [Time: 790ms] CPU Idle
 [Time: 800ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 810ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 820ms] Running: 'Temperature Sensor' (Remaining: 20ms)
 [Time: 830ms] Running: 'Temperature Sensor' (Remaining: 10ms)
 [Time: 840ms] CPU Idle
 [Time: 850ms] CPU Idle
 [Time: 860ms] CPU Idle
 [Time: 870ms] CPU Idle
 [Time: 880ms] CPU Idle
 [Time: 890ms] CPU Idle
 [Time: 900ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 910ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 920ms] CPU Idle
 [Time: 930ms] CPU Idle
 [Time: 940ms] CPU Idle
 [Time: 950ms] CPU Idle
 [Time: 960ms] CPU Idle
 [Time: 970ms] CPU Idle
 [Time: 980ms] CPU Idle
 [Time: 990ms] CPU Idle
 [Time: 1000ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 1010ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 1020ms] Running: 'Temperature Sensor' (Remaining: 20ms)
 [Time: 1030ms] Running: 'Temperature Sensor' (Remaining: 10ms)
 [Time: 1040ms] Running: 'SoC Calculator' (Remaining: 50ms)
 [Time: 1050ms] Running: 'SoC Calculator' (Remaining: 40ms)
 [Time: 1060ms] Running: 'SoC Calculator' (Remaining: 30ms)
 [Time: 1070ms] Running: 'SoC Calculator' (Remaining: 20ms)
 [Time: 1080ms] Running: 'SoC Calculator' (Remaining: 10ms)
 [Time: 1090ms] Running: 'Cell Balancer' (Remaining: 100ms)
 [Time: 1100ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 1110ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 1120ms] Running: 'Cell Balancer' (Remaining: 90ms)

[Time: 1130ms] Running: 'Cell Balancer' (Remaining: 80ms)
 [Time: 1140ms] Running: 'Cell Balancer' (Remaining: 70ms)
 [Time: 1150ms] Running: 'Cell Balancer' (Remaining: 60ms)
 [Time: 1160ms] Running: 'Cell Balancer' (Remaining: 50ms)
 [Time: 1170ms] Running: 'Cell Balancer' (Remaining: 40ms)
 [Time: 1180ms] Running: 'Cell Balancer' (Remaining: 30ms)
 [Time: 1190ms] Running: 'Cell Balancer' (Remaining: 20ms)
 [Time: 1200ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 1210ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 1220ms] Running: 'Temperature Sensor' (Remaining: 20ms)
 [Time: 1230ms] Running: 'Temperature Sensor' (Remaining: 10ms)
 [Time: 1240ms] Running: 'Cell Balancer' (Remaining: 10ms)
 [Time: 1250ms] CPU Idle
 [Time: 1260ms] CPU Idle
 [Time: 1270ms] CPU Idle
 [Time: 1280ms] CPU Idle
 [Time: 1290ms] CPU Idle
 [Time: 1300ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 1310ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 1320ms] CPU Idle
 [Time: 1330ms] CPU Idle
 [Time: 1340ms] CPU Idle
 [Time: 1350ms] CPU Idle
 [Time: 1360ms] CPU Idle
 [Time: 1370ms] CPU Idle
 [Time: 1380ms] CPU Idle
 [Time: 1390ms] CPU Idle
 [Time: 1400ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 1410ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 1420ms] Running: 'Temperature Sensor' (Remaining: 20ms)
 [Time: 1430ms] Running: 'Temperature Sensor' (Remaining: 10ms)
 [Time: 1440ms] CPU Idle
 [Time: 1450ms] CPU Idle
 [Time: 1460ms] CPU Idle
 [Time: 1470ms] CPU Idle
 [Time: 1480ms] CPU Idle
 [Time: 1490ms] CPU Idle
 [Time: 1500ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 1510ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 1520ms] Running: 'SoC Calculator' (Remaining: 50ms)
 [Time: 1530ms] Running: 'SoC Calculator' (Remaining: 40ms)
 [Time: 1540ms] Running: 'SoC Calculator' (Remaining: 30ms)
 [Time: 1550ms] Running: 'SoC Calculator' (Remaining: 20ms)
 [Time: 1560ms] Running: 'SoC Calculator' (Remaining: 10ms)
 [Time: 1570ms] CPU Idle
 [Time: 1580ms] CPU Idle
 [Time: 1590ms] CPU Idle
 [Time: 1600ms] Running: 'Voltage Monitor' (Remaining: 20ms)
 [Time: 1610ms] Running: 'Voltage Monitor' (Remaining: 10ms)
 [Time: 1620ms] Running: 'Temperature Sensor' (Remaining: 20ms)


```

[Time: 1630ms] Running: 'Temperature Sensor' (Remaining: 10ms)
[Time: 1640ms] CPU Idle
[Time: 1650ms] CPU Idle
[Time: 1660ms] CPU Idle
[Time: 1670ms] CPU Idle
[Time: 1680ms] CPU Idle
[Time: 1690ms] CPU Idle
[Time: 1700ms] Running: 'Voltage Monitor' (Remaining: 20ms)
[Time: 1710ms] Running: 'Voltage Monitor' (Remaining: 10ms)
[Time: 1720ms] CPU Idle
[Time: 1730ms] CPU Idle
[Time: 1740ms] CPU Idle
[Time: 1750ms] CPU Idle
[Time: 1760ms] CPU Idle
[Time: 1770ms] CPU Idle
[Time: 1780ms] CPU Idle
[Time: 1790ms] CPU Idle
[Time: 1800ms] Running: 'Voltage Monitor' (Remaining: 20ms)
[Time: 1810ms] Running: 'Voltage Monitor' (Remaining: 10ms)
[Time: 1820ms] Running: 'Temperature Sensor' (Remaining: 20ms)
[Time: 1830ms] Running: 'Temperature Sensor' (Remaining: 10ms)
[Time: 1840ms] CPU Idle
[Time: 1850ms] CPU Idle
[Time: 1860ms] CPU Idle
[Time: 1870ms] CPU Idle
[Time: 1880ms] CPU Idle
[Time: 1890ms] CPU Idle
[Time: 1900ms] Running: 'Voltage Monitor' (Remaining: 20ms)
[Time: 1910ms] Running: 'Voltage Monitor' (Remaining: 10ms)
[Time: 1920ms] CPU Idle
[Time: 1930ms] CPU Idle
[Time: 1940ms] CPU Idle
[Time: 1950ms] CPU Idle
[Time: 1960ms] CPU Idle
[Time: 1970ms] CPU Idle
[Time: 1980ms] CPU Idle
[Time: 1990ms] CPU Idle
Simulation Finished...

```

Final Performance Metrics Report

At 2000ms, the simulation terminates and outputs the automated summary report:

(Code Pasted from Output File)

```

(RMS PERFORMANCE METRICS)
Total Execution Ticks: 200
Idle Ticks Recorded: 90
Total Deadline Misses: 0
Overall CPU Utilization: 55%

```

Analysis

Based on the execution output, our real-time scheduler performed exactly as expected under the Rate Monotonic Scheduling rules. We can analyze the performance by looking at three main areas: deadline fulfillment, system preemption, and resource utilization.

Deadline Fulfillment

The final metrics show that the "Total Deadline Misses" is exactly 0. This means every single task completed its required execution time before its next period arrived. For example, the 'Voltage Monitor' requires 20ms of execution time every 100ms. In every 100ms window, the scheduler successfully allocated exactly two 10ms ticks to this task. Because the total workload never overwhelmed the processor, the system maintained perfect stability.

Preemption and Priority Validation

The logs clearly show priority preemption working correctly. At time 90ms, the 'Cell Balancer' (Priority 4) begins executing. However, at time 100ms, the period for the 'Voltage Monitor' (Priority 1) resets. Even though the 'Cell Balancer' still had 90ms of work left, the scheduler immediately preempted it and gave the CPU to the 'Voltage Monitor'. The 'Cell Balancer' was safely paused and only resumed later at time 120ms once the higher-priority tasks finished. This proves our priority logic successfully protects the most critical safety tasks.

Performance Bottlenecks and Idle Time

Our scheduler recorded 90 idle ticks out of 200 total ticks. This tells us that the CPU was empty and waiting for new periods to arrive for 45% of the total simulation time. While having a lot of idle time means the CPU is not fully active, in a safety-critical system like an electric vehicle BMS, this is actually a good thing. The idle time acts as a safety buffer. If a task takes slightly longer due to a hardware delay in a real world scenario, the extra idle space ensures the system can handle the unexpected workload without crashing or missing deadlines.

Conclusion

Through this project, we successfully designed and simulated a real-time, deterministic Battery Management System using Rate Monotonic Scheduling. The simulation ran for 2000ms, handling 5 concurrent battery tasks with perfect timing and zero deadline misses.

Key Learnings

We learned how a real-time operating system handles safety critical tasks using priority based preemption. We gained practical experience with POSIX threads (pthreads) in C++ to represent parallel hardware jobs. We also learned how to protect shared variables using mutex locks, which ensures that multiple tasks do not corrupt system data when checking deadlines and execution times.

Challenges Faced

The main challenge we faced was handling the timeline coordination inside our scheduler loop. Initially, ensuring that a task properly paused when preempted and smoothly resumed its remaining execution time later required careful management of our variables. We resolved this

by forcing the scheduler to evaluate all 5 tasks at every single 10ms tick, ensuring that priority math is calculated continuously without skipping any cycles.

Future Improvements

While our simulator functions reliably, several enhancements could be implemented to make it match a production-grade automotive RTOS:

- **Dynamic Execution Times:** In our current project, execution times are fixed (e.g., the SoC Calculator always takes exactly 50ms). In a real vehicle, execution times vary depending on the data. We could introduce random execution fluctuations to see how the scheduler reacts under sudden stress.
 - **Aperiodic Event Handling:** A real BMS needs to handle unexpected, random alerts like an immediate emergency stop if a collision sensor trips. We could add an asynchronous interrupt thread that completely bypasses the periodic RMS loop for instant safety shutdown.
 - **Context Switching Overhead:** Our simulator assumes that switching from one task to another takes zero time. In a real operating system, saving and loading CPU registers takes a small fraction of a millisecond. Adding a tiny timing penalty for every preemption would make our performance metrics even more accurate to real world conditions.
-